CLIMATE INTELLIGENCE

Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning

USER INTERFACE, SIMULATION AND VALIDATION

A guided case study of interaction and evidence

Student name: _____ Roll number: _____

Class / division: _____ Course: _____ Institution: _____

1 USER NEED AND APPLICATION OVERVIEW

A student or evaluator needs to move from a dashboard summary to individual observations, analysis and a repeatable warning demonstration. The local Tkinter application uses a sidebar to navigate Overview, Climate records, Analysis & alerts, Predictions and Project details. The capture below is the one genuine supplied application screenshot; its values describe that captured database state.

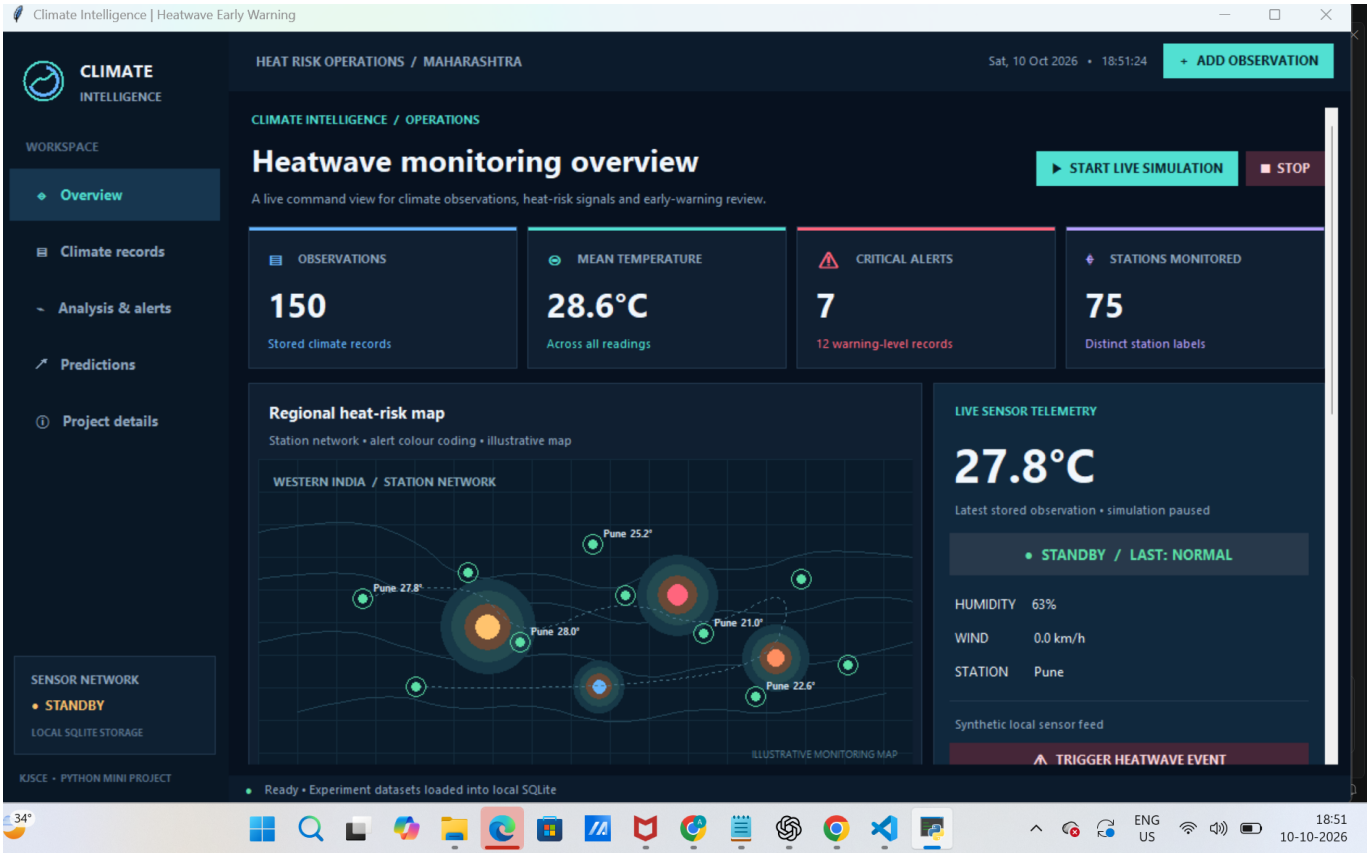


Figure 1. Supplied Overview capture: paused simulation, stored metrics and illustrative station canvas.

Objective: provide clear data entry/search, CSV exchange, climate summaries, textual warning states and a simulation that can be explained without implying remote live monitoring.

2 INTERFACE STRUCTURE AND USER WORKFLOWS

SCREEN	USER TASK / FEEDBACK
Overview	Read metric cards, latest values, illustrative map/telemetry; start/stop synthetic feed or trigger demo.
Climate records	Search by station/city/category/source; select, add, update or delete; CSV import/export.
Analysis & alerts	Review temperature/rain summaries, >=40°C count, category labels and concurrent-monitor result.
Predictions	View recent readings and one-step per-record trend estimate.
Project details	Inspect experiment, dataset and limitation information.

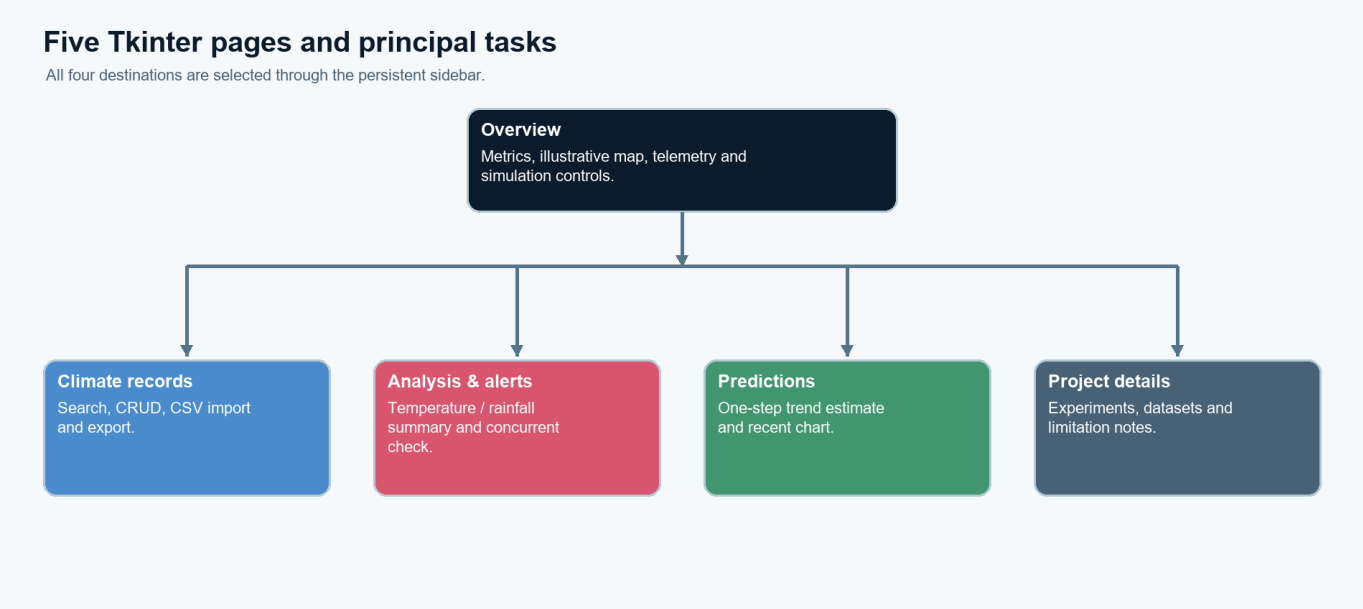


Figure 2. Page destinations are frames selected through Tkinter callbacks.

Deterministic demo action · app.py

```
record = {
 "observed_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
 "station": "Nagpur Heatwave Demo", "city": "Nagpur",
 "temperature": 41.6, "humidity": 29.0, "wind_speed": 7.5, "rainfall": 0.0,
 "condition_text": "Simulated heatwave event", "source": "Scenario simulation",
 "notes": "Synthetic demonstration event; not a live weather observation"
}
```

The database assigns the corresponding temperature category during insertion. Text accompanies color to make severity visible; accessibility and participant testing were not conducted.

3 RECORDS, DATASETS AND COURSE CONCEPTS

SUPPLIED CSV	ROWS / ROLE	FIELDS RETAINED FOR THE COMMON RECORD
weather_data exp 03.csv	50 / bootstrap	Station, city, temperature, humidity, wind, condition
Exp04_IMD_50_Real_Weather_Observations.csv	50 / bootstrap	Date, station, temperature, rainfall; source label unverified
Pune_Climate_50_Observations EXP 6.csv	50 / bootstrap	Time, station, temperature, humidity
Weather data (EXP5).csv	24,070 / manual import	Selected core weather fields from 20 columns

Two XLSX companions are supplied but import/export dialogs are CSV-only. Empty storage seeds the three 50-row files. SQLite stores one observations table (12 fields) and indexes date/time and station; CRUD is parameterized, but repeated imports may duplicate records and source-level provenance is incomplete.

EXP. / TOPIC	CONCEPT → ACTUAL APPLICATION
01 · Class creation	Classes, objects, constructor, attributes and encapsulation. Validated observation objects carry form/import values; marks classes remain outside the UI.
02 · Constructors and lists	Default/parameterized construction, list accumulation and CRUD. List-backed import staging leads to saved rows; record controls operate on persistent SQLite CRUD.
03 · String handling and inheritance	Whitespace/string methods, base and derived classes, super(). Clean labels and the imported subclass keep displayed records consistent with the base model.
04 · Abstract classes and polymorphism	ABC, abstract method, overriding and a shared method contract. Analysis screens call either analyzer through the shared method interface.
05 · Exceptions, modules and multithreading	Exceptions, module separation, background/concurrent work. Background simulation/monitoring returns queued results for Tk updates; errors are shown as feedback.
06 · Collections and functional programming	Lists, mappings, sets, lambda, map, filter and reduce. Collections and lambdas support the Analysis & alerts results users inspect in the GUI.

4 SIMULATION, RISK FEEDBACK AND IMPLEMENTATION EVIDENCE

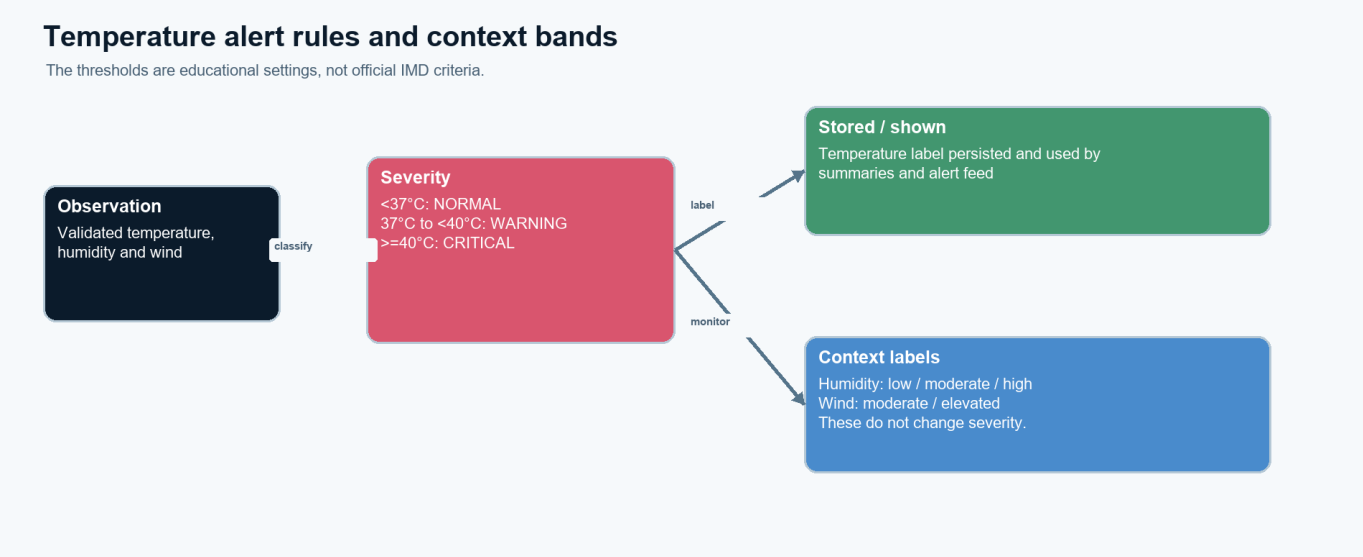


Figure 3. Stored alert bands are temperature-only; humidity/wind are separate context labels.

The live-simulation control creates synthetic local observations on a worker and returns events through a queue for Tk polling. Stop signals the worker. Heatwave Event persists a fixed 41.6°C Nagpur example. Both paths are repeatable demonstrations; neither represents measured weather.

Temperature severity · alert_engine.py

```
if temp >= CRITICAL_C:
 return "CRITICAL"
if temp >= WARNING_C:
 return "WARNING"
return "NORMAL"
```

Concurrent monitor · app.py

```
from concurrent.futures import ThreadPoolExecutor
def check(row):
 r=dict(row);r.update(monitor_conditions(r))
 return r
with ThreadPoolExecutor(max_workers=8, thread_name_prefix="heat-check") as pool:
 result=list(pool.map(check, rows))
self.events.put(("monitor_done",result))
```

The estimate shown on Predictions uses up to eight latest readings and adds one average per-reading change to the latest value. It does not account for unequal timestamp gaps and has no accuracy score. There is no trained model or external data feed.

5 OBSERVED TESTS, LIMITATIONS AND CONCLUSION

TEST AREA	VERIFIED OBSERVATION
Navigation / geometry	GUI smoke visits all five page keys and checks 880×580 construction.
Input and risk	Validation rejects invalid humidity; boundary tests pass at 36.9, 37 and 40°C.
Storage / CSV	CRUD, 150-row bootstrap and 50-row import/export round trip pass.
Demonstration	Start/stop simulation persists a reading; event stores 41.6°C as CRITICAL.
Suite total	16/16 automated tests passed on 13 Oct 2026 using temporary databases.

Limits and conclusion

The UI supports a clear classroom walkthrough, but only one authentic full-screen capture was supplied; other pages are covered by source inspection and smoke tests rather than screenshots. No usability study, screen-reader review, XLSX workflow, real sensor/API, official alert validation or forecast accuracy test is claimed. Imported, manually entered, synthetic and estimated values must remain distinct during a demonstration. Within those limits, the five-page workflow links understandable feedback with persistent local records.

Selected references

[1] Python Software Foundation. Python 3 Library Reference: tkinter, csv, sqlite3 and concurrent.futures. [2] SQLite Consortium, SQL Language Reference. [3] Supplied Python experiments 01–06 and project source files, preserved in the project workspace.